Math Templates

Quick Reference Table

#	Name	Description	Intuition	Time	Space	Key Implementation
7	Reverse Integer	Given a signed 32-bit integer `x`, return `x` with its digits reversed. Return 0 if the result overflows.	Peel the last digit with `%10` and push it onto a growing `result` with `*10`. The only hazard is overflow, so check capacity *before* the push.	O(log n)	O(1)	Overflow guard — before `result = result*10 + digit`, verify `result` is not past `Integer.MAX_VALUE/10` (or below `MIN_VALUE/10`).
9	Palindrome Number	Return true if integer `x` reads the same forwards and backwards. Negatives are never palindromes.	No need to reverse the whole number — build up the reversed second half until it meets or passes the shrinking first half. At the meeting point the two halves should match.	O(log n)	O(1)	Half reverse — loop while `x > reversed`; the middle digit (odd-length case) is dropped via `reversed / 10`.
50	Pow(x, n)	Implement `pow(x, n)`, computing `x` raised to the integer power `n`.	Squaring the base doubles the exponent reach, so each bit of `n` decides whether the current power of `x` joins the product — O(log n) multiplications instead of O(n).	O(log n)	O(1)	Handle negative exponent by inverting `x` and negating `N` (use `long` so `-Integer.MIN_VALUE` doesn't overflow); multiply into `result` only when the current exponent bit is set.
372	Super Pow	Compute `a^b mod 1337` where `b` is given as an array of digits (so `b` may be astronomically large).	Exponents add when powers multiply, and reading `b` digit by digit means `a^b = (a^(b/10))^10 * a^(b%10)`. Fold left across the digits, applying the modulus at every step to keep numbers small.	O(log n)	O(1)	Digit-by-digit modular exponentiation — at each new digit `d`, raise the running result to the 10th power and multiply by `a^d`, all mod 1337.
168	Excel Sheet Column Title	Given a column number, return its Excel-style title (1→"A", 27→"AA", 28→"AB").	It's base-26, but Excel has no zero digit — columns start at A=1, so the system is 1-indexed (a "bijective base 26"). Decrement before each digit to shift into the standard 0..25 range.	O(log n)	O(1)	1-indexed — do `columnNumber--` before extracting each digit so that `% 26` yields 0..25 mapping cleanly onto 'A'..'Z'.
171	Excel Sheet Column Number	Given an Excel column title (e.g. "AB"), return its column number (28).	Horner's rule for base-26, but each letter contributes `c-'A'+1` (A=1, not 0) because the system is 1-indexed.	O(L) L = title length	O(1)	1-indexed letters — `(c - 'A' + 1)` so 'A' maps to 1 rather than 0.
67	Add Binary	Given two binary strings `a` and `b`, return their sum as a binary string.	Grade-school addition from the rightmost digit, tracking a carry — identical to adding two linked-list numbers (#2), but the base is 2 and the operands are strings.	O(n) n = max length	O(n)	Char arithmetic per column — `(a.charAt(i)-'0') + (b.charAt(j)-'0') + carry`; emit `sum % 2` and carry `sum / 2`.
204	Count Primes	Count the number of prime numbers strictly less than `n`.	Instead of testing each number for primality, cross out every multiple of each prime once. When you reach an unmarked `i`, it's prime; its multiples below `i*i` were already crossed out by smaller primes, so start marking at `i*i`.	O(n log log n)	O(n)	Start marking at `i*i` — multiples `k*i` with `k < i` were already handled by the smaller factor `k`.
1201	Ugly Number III	Find the nth positive integer that is divisible by at least one of `a`, `b`, or `c`.	The count of valid numbers ≤ `x` is monotonic in `x`, so binary-search the smallest `x` whose count reaches `n`. Counting "≤ mid divisible by a, b, or c" is a classic inclusion-exclusion over the three sets, subtracting pairwise LCM overlaps and adding back the triple LCM.	O(log min(range))	O(1)	Inclusion-exclusion — `count = mid/a + mid/b + mid/c - mid/ab - mid/bc - mid/ac + mid/abc` using LCMs to avoid double counting.
29	Divide Two Integers	Divide two integers `dividend` and `divisor` without using multiplication, division, or mod, truncating toward zero. Clamp the result to the 32-bit signed range.	Repeated subtraction is too slow, so subtract the largest doubling of the divisor that still fits — this is long division in binary. Work in negatives so `Integer.MIN_VALUE` cannot overflow.	O(log n * log n)	O(1)	Exponential search of the divisor — double `divisor` and the matching quotient bit until it would exceed the remaining dividend.
59	Spiral Matrix II	Given `n`, generate an `n x n` matrix filled with the numbers `1` to `n*n` in spiral order.	Walk the four borders inward, shrinking the boundary after completing each side, filling a running counter.	O(n^2)	O(1) extra	Layer boundaries — maintain `top/bottom/left/right` and fill right, down, left, up in turn.
66	Plus One	Given a large integer represented as an array of digits (most significant first), increment it by one and return the resulting digit array.	Add one from the rightmost digit; a digit below 9 absorbs the carry and we're done, otherwise it becomes 0 and the carry propagates. If every digit was 9 we need one extra leading digit.	O(n)	O(1) extra (O(n) only for all-nines)	Early return — the moment a digit is incremented without rolling over, return; only an all-9 array needs a longer result.
118	Pascal's Triangle	Given `numRows`, generate the first `numRows` rows of Pascal's triangle.	Each interior entry is the sum of the two entries above it; the edges are always 1.	O(numRows^2)	O(numRows^2)	Row-by-row build — `row[j] = previous[j-1] + previous[j]`.
119	Pascal's Triangle II	Given a row index `rowIndex`, return that row of Pascal's triangle using only O(rowIndex) extra space.	Update a single row in place; iterating right-to-left lets each entry read its left neighbor before that neighbor is overwritten.	O(rowIndex^2)	O(rowIndex)	In-place reverse update — `row[j] += row[j-1]` scanning from the right.
149	Max Points on a Line	Given an array of points on a plane, return the maximum number of points lying on the same straight line.	Fix one point as the anchor; every other point defines a slope from it, and collinear points share that slope. Count the most common slope per anchor, using a	O(n^2)	O(n)	Slope histogram per anchor — key on `(dy/g, dx/g)` reduced by gcd, with sign normalization.

#	Name	Description	Intuition	Time	Space	Key Implementation
			reduced integer fraction as the key to avoid floating-point error.			
172	Factorial Trailing Zeroes	Given an integer `n`, return the number of trailing zeroes in `n!`.	A trailing zero comes from a factor of 10 = 2*5, and factors of 2 are far more plentiful than factors of 5, so just count the factors of 5. Multiples of 25 contribute an extra 5, of 125 another, and so on.	O(log n)	O(1)	Count factors of 5 — sum `n/5 + n/25 + n/125 + ...` via `n /= 5`.
202	Happy Number	A happy number reaches 1 by repeatedly replacing it with the sum of the squares of its digits; return whether `n` is happy.	The sequence either hits 1 or enters a cycle, so this is cycle detection — use Floyd's slow/fast pointers over the "sum of digit squares" transformation.	O(log n) per step	O(1)	Floyd cycle detection on a number transform — `slow = f(slow)`, `fast = f(f(fast))`.
223	Rectangle Area	Given the coordinates of two axis-aligned rectangles, return the total area they cover (counting the overlap once).	Total area is the sum of both areas minus the overlap; the overlap is itself a rectangle whose width and height are the intersections of the projections onto each axis (zero if they don't intersect).	O(1)	O(1)	Inclusion-exclusion of areas — overlap width = `min(rights) - max(lefts)` clamped at 0.
233	Number of Digit One	Given an integer `n`, count the total number of digit `1` appearing in all non-negative integers from 0 to `n`.	Count contributions of the digit 1 at each place value separately. For a given place, the count depends on the higher digits, the current digit, and the lower digits, which split cleanly into full cycles plus a partial cycle.	O(log n)	O(1)	Per-place digit counting — for each power-of-ten place, `count += high*place + adjust(cur, low, place)`.
243	Shortest Word Distance	Given an array of words and two distinct words, return the shortest distance (in indices) between any occurrence of the two words.	A single pass tracking the most recent index of each target word suffices; whenever both have been seen, the gap between the two latest positions is a candidate minimum.	O(n)	O(1)	Two latest-index trackers — update `result` whenever both indices are valid.
258	Add Digits	Repeatedly add all the digits of a non-negative integer until the result has a single digit; return it (the digital root).	The digital root has a closed form derived from numbers being congruent to their digit sum modulo 9: the answer is `0` for 0, otherwise `1 + (n−1) % 9`.	O(1)	O(1)	Digital root formula — no loop, just `1 + (num - 1) % 9`.
263	Ugly Number	An ugly number has no prime factors other than 2, 3, or 5; return whether `n` is ugly.	Divide out all factors of 2, 3, and 5; what remains is 1 exactly when no other prime factor exists. Non-positive numbers are never ugly.	O(log n)	O(1)	Factor stripping — repeatedly divide by each of 2, 3, 5 and check the remainder is 1.
292	Nim Game	You and an opponent alternate removing 1 to 3 stones; whoever takes the last stone wins. Given `n` stones and you move first, return whether you can win.	If `n` is a multiple of 4 you always lose, because whatever you take (1-3) the opponent can complete a group of four, keeping `n` a multiple of 4 on your turn until 0.	O(1)	O(1)	Multiple-of-four loss — win exactly when `n % 4 != 0`.
319	Bulb Switcher	There are `n` bulbs initially off; in round `i` you toggle every `i`-th bulb. Return how many bulbs are on after `n` rounds.	Bulb `k` is toggled once per divisor of `k`, ending on only if it has an odd number of divisors — which happens exactly for perfect squares. The count of perfect squares up to `n` is `floor(sqrt(n))`.	O(1)	O(1)	Perfect-square count — answer is `(int) sqrt(n)`.
326	Power of Three	Given an integer `n`, return whether it is a power of three.	Within the 32-bit range the largest power of three is `3^19 = 1162261467`; any power of three must divide it exactly, so a single divisibility test works.	O(1)	O(1)	Max-power divisibility — `n > 0 && 1162261467 % n == 0`.
357	Count Numbers with Unique Digits	Given `n`, count all numbers `x` with `0 <= x < 10^n` that have no repeated digits.	Count by length: the first digit has 9 choices (1-9), the next 9 (any but the first), then 8, 7, ..., multiplying as digits are added. Sum these counts across lengths 1 to n, plus the single number 0.	O(n)	O(1)	Permutation counting per length — `9 * 9 * 8 * ...` accumulated.
365	Water and Jug Problem	Given two jugs of capacities `x` and `y` and a target `target`, determine if you can measure exactly `target` liters using fill/empty/pour operations.	Any amount measurable is a non-negative integer combination of `x` and `y`, which by Bezout's identity is exactly the multiples of `gcd(x, y)`. So the target must be reachable in total capacity and divisible by the gcd.	O(log(min(x,y)))	O(1)	Bezout / gcd test — `target % gcd(x, y) == 0` with `target <= x + y`.
367	Valid Perfect Square	Given a positive integer `num`, return whether it is a perfect square, without using any built-in sqrt function.	The square root lies in `[1, num]` and squaring is monotonic, so binary search the candidate root and compare its square (in `long` to avoid overflow) against `num`.	O(log n)	O(1)	Binary search on the root — midpoint `k = i + (j−i)/2`, compare `k*k` to `num`.
397	Integer Replacement	Given a positive integer `n`, each step you may replace it with `n/2` if even, or `n+1` / `n−1` if odd; return the minimum steps to reach 1.	Halving is always best when possible. For odd `n`, choosing `+1` vs `−1` should expose more trailing zeros to halve next; `n == 3` is the special case where `−1` is better.	O(log n)	O(1)	Greedy on the last two bits — for odd `n != 3`, add 1 when `(n & 2) != 0` else subtract 1.
398	Random Pick Index	Given an array `nums`, implement `pick(target)` returning a uniformly random index where `nums[index] == target`.	Reservoir sampling lets us pick uniformly in one pass without storing all matching indices: the `k`-th matching element replaces the current pick with probability `1/k`.	O(n) per pick	O(1)	Reservoir sampling of size 1 — keep the index with probability `1/count` as matches stream by.
400	Nth Digit	Given `n`, return the `n`-th digit in the infinite sequence of concatenated positive integers `123456789101112...`.	Numbers group by digit-length: there are `9*10^(d−1)` numbers with `d` digits, contributing `d * 9 * 10^(d−1)` digits. Subtract whole groups to find the length, locate the exact number, then index into it.	O(log n)	O(1)	Length-bucket walk — subtract `count * digitLength` until `n` falls inside the current bucket.
412	Fizz Buzz	Return a list of strings for `1..n` where multiples of 3 become "Fizz", multiples of 5 become "Buzz", multiples of both	Build each entry by concatenating "Fizz" and/or "Buzz" based on divisibility; if neither applies, use the number's string.	O(n)	O(1) extra	Concatenate divisibility tags — append "Fizz" on `%3`, "Buzz" on `%5`.

#	Name	Description	Intuition	Time	Space	Key Implementation
		become "FizzBuzz", and others the number itself.				
441	Arranging Coins	You have `n` coins to form a staircase where the `k` -th row has exactly `k` coins; return the number of complete rows.	After `k` complete rows you've used `k(k+1)/2` coins, so the answer is the largest `k` with `k(k+1)/2 <= n` . Binary search this `k` (or solve the quadratic).	O(log n)	O(1)	Binary search on rows — compare triangular number `k(k+1)/2` against `n` using `long` .
453	Minimum Moves to Equal Array Elements	In one move you increment `n−1` elements of the array by 1; return the minimum moves to make all elements equal.	Incrementing all but one is equivalent (for the purpose of equalizing) to decrementing a single element by 1. So the answer is the total amount each element exceeds the minimum: `sum − n * min` .	O(n)	O(1)	Relative-decrement reframing — total moves = `sum(nums) − n * min(nums)` .
458	Poor Pigs	With `buckets` buckets one of which is poisoned, `minutesToDie` for poison to act, and `minutesToTest` total time, return the minimum pigs needed to find the poisoned bucket.	Each pig can be tested `t = minutesToTest / minutesToDie` times, so it has `t + 1` distinguishable states (died after round 1..t, or survived). With `p` pigs we cover `(t+1)^p` buckets, so we need the smallest `p` with `(t+1)^p >= buckets` .	O(log buckets)	O(1)	Base-(tests+1) counting — `p = ceil(log_{t+1}(buckets))` .
470	Implement Rand10() Using Rand7()	Given `rand7()` uniform in [1,7], implement `rand10()` uniform in [1,10].	Two calls form a uniform value in [1,49]; reject anything above 40 and map the remaining 40 outcomes evenly onto [1,10]. Rejection keeps the distribution exactly uniform.	O(1) expected	O(1)	Rejection sampling on a 7x7 grid — accept index `< 40` , return `index % 10 + 1` .
492	Construct the Rectangle	Given a target `area` , return the dimensions `[L, W]` with `L >= W` , `L * W == area` , and `L − W` minimized.	The most square-like factor pair minimizes the difference, so start `W` at `floor(sqrt(area))` and walk down until it divides `area` .	O(sqrt(area))	O(1)	Search down from sqrt — first divisor `W <= sqrt(area)` gives the closest pair.
495	Teemo Attacking	Given sorted attack `timeSeries` and a poison `duration` , return the total time the target is poisoned (overlaps counted once).	Each attack would add `duration` , but if the next attack comes before the current poison ends, only the gap between attacks counts. Sum the capped gaps and add a full duration for the last attack.	O(n)	O(1)	Capped gaps — add `min(gap, duration)` between consecutive attacks.
528	Random Pick with Weight	Given an array `w` of weights, implement `pickIndex()` returning index `i` with probability proportional to `w[i]` .	Build prefix sums so the cumulative weights partition `[1, total]` into intervals; draw a random target in that range and binary-search the first prefix sum reaching it.	O(log n) per pick, O(n) build	O(n)	Prefix-sum + binary search — find leftmost prefix `>= target` .
593	Valid Square	Given four points, return whether they form a valid square (positive area).	Among the six pairwise squared distances of a square there are exactly two distinct values: four equal sides and two equal (larger) diagonals, with the diagonal twice the side. Use squared distances to stay in integers.	O(1)	O(1)	Two-distinct-distance check — collect six squared distances; require exactly two values, the smaller nonzero, the larger double it.
598	Range Addition II	Given an `m x n` matrix of zeros and operations each incrementing the top-left `a x b` submatrix, return how many cells hold the maximum value.	Every operation always covers cell (0,0), so the maximum value sits in the intersection of all operation rectangles — its area is the product of the smallest row bound and smallest column bound.	O(ops)	O(1)	Intersection area of all ops — `min(all a) * min(all b)` .
633	Sum of Square Numbers	Given a non-negative integer `c` , decide whether there exist non-negative integers `a, b` with `a*a + b*b == c` .	Squeeze two pointers from `0` and `floor(sqrt(c))` : if the squared sum is too small move the low pointer up, too large move the high pointer down.	O(sqrt(c))	O(1)	Two pointers on squares — `a` from 0 up, `b` from sqrt(c) down.
656	Coin Path	Given `coins` (cost per index, `−1` blocked) and a max jump `maxJump` , find the lexicographically smallest cheapest path of indices from index 0 to the last.	Work backwards with DP: the best cost from index `i` is its cost plus the cheapest reachable next index within `maxJump` . Filling from the right and preferring the smaller next index yields the lexicographically smallest path on ties.	O(n * maxJump)	O(n)	Backward DP with path reconstruction — `cost[i] = coins[i] + min over j in (i, i+maxJump]` , tie-break to smaller `j` .
781	Rabbits in Forest	Each surveyed rabbit reports how many other rabbits share its color; given the `answers` , return the minimum number of rabbits in the forest.	Rabbits answering `k` form groups of size `k+1` . For each answer value, every full group of `k+1` such replies fills one color group; partial groups still require a whole `k+1` block.	O(n)	O(n)	Group-rounding by answer — for count `c` of answer `k` , add `ceil(c/(k+1)) * (k+1)` .
789	Escape The Ghosts	Starting at the origin with a `target` , and given `ghosts` positions, you all move simultaneously in Manhattan steps; return whether you can reach the target before any ghost catches you.	You win iff you reach the target strictly before every ghost. Since all move optimally with Manhattan distance, compare your distance from origin to each ghost's distance to the target; if no ghost is at least as close, you escape.	O(g)	O(1)	Manhattan-distance race — you win when your distance to target < every ghost's distance to target.
792	Number of Matching Subsequences	Given a string `s` and an array of `words` , return how many words are subsequences of `s` .	Rather than scan `s` per word, bucket words by their next-needed character. Sweep `s` once; each character releases its waiting words, advancing them to their next character bucket, and any word that runs out of characters is a match.	O(|s| + total word length)	O(total word length)	Waiting lists per next-char — advance words bucketed by the character they currently need as `s` streams.
829	Consecutive Numbers Sum	Given `n` , return the number of ways to write it as a sum of consecutive positive integers.	A run of `k` consecutive integers starting at `a` sums to `k*a + k(k−1)/2` , so `n − k(k−1)/2` must be positive and divisible by `k` . Try each `k` while the triangular offset stays below `n` .	O(sqrt(n))	O(1)	Count valid run-lengths — for each `k` , valid iff `(n − k(k−1)/2) % k == 0` and positive.

#	Name	Description	Intuition	Time	Space	Key Implementation
836	Rectangle Overlap	Given two axis-aligned rectangles `rec1` and `rec2` as `[x1,y1,x2,y2]`, return whether they overlap in positive area.	Two rectangles overlap iff their projections on both axes overlap; equivalently, neither is entirely to one side of the other on either axis.	O(1)	O(1)	Separating-axis on both projections — overlap iff `x` ranges and `y` ranges each intersect.
858	Mirror Reflection	A square room with mirrored walls has receptors at corners 0,1,2; a laser leaves the southwest corner and first travels to the east wall at height `q` (room side `p`). Return which receptor it first meets.	Unfold the reflections so the beam travels straight; it hits a corner after going up a multiple of `p` that is also a multiple of `q`, i.e. `lcm(p,q)`. The parities of how many room-widths and room-heights that represents determine the receptor.	O(log(min(p,q)))	O(1)	Parity of `lcm/p` and `lcm/q` — reduce `p,q` by gcd, then the receptor follows from which is odd/even.
869	Reordered Power of 2	Given an integer `n`, return whether some permutation of its digits (no leading zero) equals a power of 2.	Two numbers are digit-permutations iff they have the same multiset of digits. So compute a digit-count signature of `n` and compare it against the signatures of all powers of 2 within the 32-bit range.	O(log^2 n)	O(1)	Digit-count signature match — compare sorted-digit fingerprint against each power of 2.
939	Minimum Area Rectangle	Given points in the plane, find the minimum area of an axis-aligned rectangle formed by four of them, or 0 if none exists.	A rectangle is fixed by its diagonal corners; for each pair of points with different x and y, the other two corners are determined, so check whether both exist in a point set.	O(n^2)	O(n)	Diagonal-pair lookup — for each pair, test if the complementary corners are present in a hash set.
963	Minimum Area Rectangle II	Given points in the plane, find the minimum area of any rectangle (any orientation) formed by four of them, or 0.	A rectangle's diagonals share the same midpoint and the same length. Group point pairs by `(midpoint, diagonal length)`; any two pairs in a group are the two diagonals of a rectangle, whose sides come from the corner vectors.	O(n^2) groups, worst O(n^2) per group	O(n^2)	Group diagonals by midpoint+length — within a group, combine pairs and compute area via vectors.
1041	Robot Bounded In Circle	A robot starts at the origin facing north and repeats `instructions` ('G' forward, 'L'/'R' turn) forever; return whether it stays within some bounded circle.	After one pass, the robot is bounded iff it returns to the origin, or it no longer faces north — a non-north heading guarantees the net displacement rotates and cancels over at most four cycles.	O(n)	O(1)	One-cycle check — bounded iff back at origin OR final direction != initial north.
1160	Find Words That Can Be Formed by Characters	Given `words` and a string `chars`, return the total length of words that can be formed using letters of `chars` (each letter used at most as often as it appears).	Count available letters once; a word is formable iff its own letter counts never exceed the available counts. Sum lengths of formable words.	O(total characters)	O(1)	Frequency containment — compare per-word letter counts against the `chars` budget.
1304	Find N Unique Integers Sum up to Zero	Given `n`, return any array of `n` unique integers that sum to zero.	Pair each positive `i` with its negation `-i`; that cancels to zero, and add a lone 0 if `n` is odd.	O(n)	O(1) extra	Symmetric pairs — emit `i` and `-i`, plus a central 0 for odd `n`.
1344	Angle Between Hands of a Clock	Given an `hour` and `minutes`, return the smaller angle (in degrees) between the hour and minute hands.	The minute hand moves 6 degrees per minute; the hour hand moves 30 degrees per hour plus 0.5 degree per minute. The answer is the absolute difference, folded to at most 180.	O(1)	O(1)	Per-hand angular position — minute = `6*m`, hour = `30*(h%12) + 0.5*m`, take `min(diff, 360-diff)`.
1583	Count Unhappy Friends	Given `n` friends, their `preferences`, and a `pairs` assignment, count friends who are unhappy — paired with someone they prefer less than another friend who also prefers them over their own partner.	Precompute each friend's preference rank for every other friend. A friend `x` (paired with `y`) is unhappy if some `u` ranks higher than `y` for `x`, and `u` ranks `x` higher than `u`'s own partner. Compare ranks pairwise.	O(n^2)	O(n^2)	Rank-matrix mutual-preference scan — `x` unhappy if exists `u` with `rank[x][u] < rank[x][y]` and `rank[u][x] < rank[u][partner[u]]`.
1588	Sum of All Odd Length Subarrays	Given an array `arr`, return the sum of all elements over all odd-length contiguous subarrays.	Instead of enumerating subarrays, count how many odd-length subarrays include each index. With `i+1` choices for the left boundary and `n-i` for the right, the number of odd-length ones is computed in closed form, weighting each element.	O(n)	O(1)	Per-element contribution — element `i` appears in `ceil(((i+1)*(n-i))/2)` odd-length subarrays.
1646	Get Maximum in Generated Array	Build array `nums` where `nums[0]=0`, `nums[1]=1`, `nums[2i]=nums[i]`, `nums[2i+1]=nums[i]+nums[i+1]`; return its maximum for size `n`.	Generate the array directly from the recurrence, tracking the running maximum. Handle the tiny base cases for `n < 2`.	O(n)	O(n)	Direct recurrence fill — even index copies, odd index sums the two halves.
1823	Find the Winner of the Circular Game	`n` friends in a circle eliminate every `k`-th friend repeatedly; return the 1-indexed winner (Josephus problem).	The Josephus recurrence builds the survivor's position from a circle of size 1 upward: the winner in a circle of `i` is `(winner_{i-1} + k) % i`. Convert the final 0-indexed answer to 1-indexed.	O(n)	O(1)	Josephus recurrence — `winner = (winner + k) % i` for `i = 2..n`.
1969	Minimum Non-Zero Product of the Array Elements	For the array of all integers `1..2^p - 1`, you may swap bits between elements any number of times; return the minimum non-zero product modulo `1e9+7`.	Pair the largest value `2^p - 1` (kept whole) with the rest: each other pair can be made `(2^p - 2)` and `1`, so the product is `(2^p - 1) * (2^p - 2)^(2^(p-1) - 1)` — compute with modular fast	O(p)	O(1)	Pairing + modular fast power — `(max) * pow(max-1, half-1) mod M`.

#	Name	Description	Intuition	Time	Space	Key Implementation
			power, but the base of the exponent must use the true (non-reduced) count.			

Canonical Templates

```
// MENTAL MODEL: number = sequence of digits in some base; iterate by repeatedly
//               peeling the last digit (% base) and shifting (/ base).
// WHEN: "reverse digits", "x^n", "base conversion", "count primes", "nth divisible-by".

// —— DIGIT EXTRACTION LOOP —— peel digits right-to-left
while (n != 0) {
    int digit = n % 10;   // last digit
    n /= 10;              // drop last digit
    // ... use digit ...
}

// —— FAST POWER (exponentiation by squaring) ——
// WHY: x^n needs only O(log n) multiplications by squaring the base
//      and consuming one exponent bit per step.
//   x^13 = x^8 * x^4 * x^1   (13 = 1101 in binary)
double fastPow(double x, long n) {
    double result = 1;
    while (n > 0) {
        if ((n & 1) == 1) result *= x;   // bit set → include this power of x
        x *= x;                          // square the base for the next bit
        n >>= 1;                         // consume one exponent bit
    }
    return result;
}

// —— BASE CONVERSION —— generic base b, 0-indexed
// number → digits: peel with % b, shift with / b, collect, then reverse
StringBuilder builder = new StringBuilder();
while (num > 0) {
    builder.append(num % b);
    num /= b;
}
builder.reverse();
// digits → number: Horner's rule
int value = 0;
for (int digit : digits) value = value * b + digit;

// —— SIEVE OF ERATOSTHENES —— mark composites up to n
// WHY start at i*i: any multiple k*i with k < i was already marked by k.
boolean[] composite = new boolean[n];
for (int i = 2; i * i < n; i++) {
    if (composite[i]) continue;
    for (int j = i * i; j < n; j += i)   // start at i*i, step by i
        composite[j] = true;
}
```

Part 1 — Digit Manipulation

#7 Reverse Integer

Description: Given a signed 32-bit integer `x`, return `x` with its digits reversed. Return 0 if the result overflows.
Intuition: Peel the last digit with `%10` and push it onto a growing `result` with `*10`. The only hazard is overflow, so check capacity *before* the push.
Variation: Overflow guard — before `result = result*10 + digit`, verify `result` is not past `Integer.MAX_VALUE/10` (or below `MIN_VALUE/10`).

```
class Solution {
    public int reverse(int x) {
        int result = 0;
        while (x != 0) {
            int digit = x % 10;
            x /= 10;
            if (result > Integer.MAX_VALUE/10 || result < Integer.MIN_VALUE/10) return 0;  // ← VARIATION: overflow guard before pus
            result = result * 10 + digit;
        }
        return result;
    }
}
```

Time O(log n) | **Space** O(1)

#9 Palindrome Number

Description: Return true if integer `x` reads the same forwards and backwards. Negatives are never palindromes.
Intuition: No need to reverse the whole number — build up the reversed second half until it meets or passes the shrinking first half. At the meeting point the two halves should match.
Variation: Half reverse — loop while `x > reversed`; the middle digit (odd-length case) is dropped via `reversed / 10`.

```
class Solution {
    public boolean isPalindrome(int x) {
        if (x < 0 || (x % 10 == 0 && x != 0)) return false;   // negatives + trailing zero can't be palindrome
        int reversed = 0;
        while (x > reversed) {                  // ← VARIATION: only reverse half the digits
            reversed = reversed * 10 + x % 10;
            x /= 10;
        }
        return x == reversed || x == reversed / 10;  // even length / odd length (drop middle digit)
    }
}
```

Time O(log n) | **Space** O(1)

Part 2 — Fast Power

#50 Pow(x, n)

Description: Implement `pow(x, n)`, computing `x` raised to the integer power `n`.
Intuition: Squaring the base doubles the exponent reach, so each bit of `n` decides whether the current power of `x` joins the product — O(log n) multiplications instead of O(n).
Variation: Handle negative exponent by inverting `x` and negating `N` (use `long` so `-Integer.MIN_VALUE` doesn't overflow); multiply into `result` only when the current exponent bit is set.

```
class Solution {
    public double myPow(double x, int n) {
        long N = n;
        if (N < 0) { x = 1 / x; N = -N; }       // ← VARIATION: handle negative exponent (long avoids overflow)
        double result = 1;
        while (N > 0) {
            if ((N & 1) == 1) result *= x;      // ← VARIATION: multiply when current bit set
            x *= x;                             // square base for next bit
            N >>= 1;
        }
        return result;
    }
}
```

Time O(log n) | **Space** O(1)

#372 Super Pow

Description: Compute `a^b mod 1337` where `b` is given as an array of digits (so `b` may be astronomically large).
Intuition: Exponents add when powers multiply, and reading `b` digit by digit means `a^b = (a^(b/10))^10 * a^(b%10)`. Fold left across the digits, applying the modulus at every step to keep numbers small.
Variation: Digit-by-digit modular exponentiation — at each new digit `d`, raise the running result to the 10th power and multiply by `a^d`, all mod 1337.

```
class Solution {
    private static final int MOD = 1337;
    public int superPow(int a, int[] b) {
        a %= MOD;
        int result = 1;
        for (int digit : b)
            result = powmod(result, 10) * powmod(a, digit) % MOD;  // ← VARIATION: digit-by-digit, (previous)^10 * a^digit
        return result;
    }
    private int powmod(int x, int k) {
        x %= MOD;
        int result = 1;
        for (int i = 0; i < k; i++) result = result * x % MOD;     // k <= 10, so plain loop is fine
        return result;
    }
}
```

Time O(log n) | **Space** O(1)

Part 3 — Base Conversion

#168 Excel Sheet Column Title

Description: Given a column number, return its Excel-style title (1→"A", 27→"AA", 28→"AB").
Intuition: It's base-26, but Excel has no zero digit — columns start at A=1, so the system is 1-indexed (a "bijective base 26"). Decrement before each digit to shift into the standard 0..25 range.
Variation: 1-indexed — do `columnNumber--` before extracting each digit so that `% 26` yields 0..25 mapping cleanly onto 'A'..'Z'.

```
class Solution {
    public String convertToTitle(int columnNumber) {
        StringBuilder builder = new StringBuilder();
        while (columnNumber > 0) {
            columnNumber--;                     // ← VARIATION: 1-indexed → shift to 0-indexed
            builder.append((char)('A' + columnNumber % 26));
            columnNumber /= 26;
        }
        return builder.reverse().toString();
    }
}
```

Time O(log n) | **Space** O(1)

#171 Excel Sheet Column Number

Description: Given an Excel column title (e.g. "AB"), return its column number (28).
Intuition: Horner's rule for base-26, but each letter contributes `c-'A'+1` (A=1, not 0) because the system is 1-indexed.
Variation: 1-indexed letters — `(c - 'A' + 1)` so 'A' maps to 1 rather than 0.

```
class Solution {
    public int titleToNumber(String columnTitle) {
        int result = 0;
        for (char c : columnTitle.toCharArray())
            result = result * 26 + (c - 'A' + 1);   // ← VARIATION: 1-indexed letters (+1)
        return result;
    }
}
```

Time O(L) L = title length | **Space** O(1)

Part 4 — Arithmetic on Strings

#67 Add Binary

Description: Given two binary strings `a` and `b`, return their sum as a binary string.
Intuition: Grade-school addition from the rightmost digit, tracking a carry — identical to adding two linked-list numbers (#2), but the base is 2 and the operands are strings.
Variation: Char arithmetic per column — `(a.charAt(i)-'0') + (b.charAt(j)-'0') + carry`; emit `sum % 2` and carry `sum / 2`.

```
class Solution {
    public String addBinary(String a, String b) {
        StringBuilder builder = new StringBuilder();
        int i = a.length() - 1, j = b.length() - 1, carry = 0;
        while (i >= 0 || j >= 0 || carry != 0) {
            int sum = carry;
            if (i >= 0) sum += a.charAt(i--) - '0';   // ← VARIATION: char → digit
            if (j >= 0) sum += b.charAt(j--) - '0';
            builder.append(sum % 2);                  // ← VARIATION: base-2 digit (sum % 2)
            carry = sum / 2;                     // base-2 carry (sum / 2)
        }
        return builder.reverse().toString();
    }
}
```

Time O(n) n = max length | **Space** O(n)

Part 5 — Sieve

#204 Count Primes

Description: Count the number of prime numbers strictly less than `n`.
Intuition: Instead of testing each number for primality, cross out every multiple of each prime once. When you reach an unmarked `i`, it's prime; its multiples below `i*i` were already crossed out by smaller primes, so start marking at `i*i`.
Variation: Start marking at `i*i` — multiples `k*i` with `k < i` were already handled by the smaller factor `k`.

```
class Solution {
    public int countPrimes(int n) {
        if (n < 2) return 0;
        boolean[] composite = new boolean[n];
        int count = 0;
        for (int i = 2; i < n; i++) {
            if (composite[i]) continue;          // already marked → not prime
            count++;
            for (int j = (long)i*i < n ? i*i : n; j < n; j += i)  // ← VARIATION: start at i*i (guard i*i overflow)
                composite[j] = true;
        }
        return count;
    }
}
```

Time O(n log log n) | **Space** O(n)

Part 6 — Binary Search on the Answer

#1201 Ugly Number III

Description: Find the nth positive integer that is divisible by at least one of `a`, `b`, or `c`.
Intuition: The count of valid numbers ≤ `x` is monotonic in `x`, so binary-search the smallest `x` whose count reaches `n`. Counting "≤ mid divisible by a, b, or c" is a classic inclusion-exclusion over the three sets, subtracting pairwise LCM overlaps and adding back the triple LCM.
Variation: Inclusion-exclusion — `count = mid/a + mid/b + mid/c − mid/ab − mid/bc − mid/ac + mid/abc` using LCMs to avoid double counting.

```
class Solution {
    public int nthUglyNumber(int n, int a, int b, int c) {
        long lo = 1, hi = 2_000_000_000L;
        long ab = lcm(a, b), bc = lcm(b, c), ac = lcm(a, c), abc = lcm(ab, c);
        while (lo < hi) {
            long mid = lo + (hi - lo) / 2;
            long count = mid/a + mid/b + mid/c - mid/ab - mid/bc - mid/ac + mid/abc;  // ← VARIATION: inclusion-exclusion via LCMs
            if (count < n) {
                lo = mid + 1;        // not enough yet → search higher
            } else {
                hi = mid;            // enough → mid might be the answer
            }
        }
        return (int) lo;
    }
    private long gcd(long x, long y) { return y == 0 ? x : gcd(y, x % y); }
    private long lcm(long x, long y) { return x / gcd(x, y) * y; }   // divide first to avoid overflow
}
```

Time O(log min(range)) | **Space** O(1)

Math Cheat Sheet

```
n % 10              // last decimal digit
n /= 10             // drop last digit
result * 10 + digit  // push a digit (build number left-to-right)
(n & 1) == 1        // odd
(n & 1) == 0        // even
n >>= 1             // halve (consume one bit)
x *= x              // square base in fast power
c - '0'             // char → digit value
c - 'A' + 1         // Excel letter → 1-indexed value
(char)('A' + d)     // 0..25 → 'A'..'Z'
i * i               // sieve marking start point
x / gcd(x,y) * y    // LCM without overflow (divide first)
```

Pattern Chooser

Signal	Technique	Time
"reverse the digits of a number"	Digit pop (`%10`) / push (`*10`) loop	O(log n)
"is the number a palindrome?"	Reverse only half, compare halves	O(log n)
"compute x^n / a^b efficiently"	Exponentiation by squaring (binary exp)	O(log n)
"x^huge mod m" / exponent given as array	Digit-by-digit modular fast power	O(log n)
"add two big numbers as strings"	Right-to-left add with carry	O(n)
"convert column number ↔ Excel title"	Base-26 conversion, **1-indexed**	O(log n) / O(L)
"convert between number and base-b string"	Horner (parse) / peel-and-reverse (build)	O(L)
"count primes below n"	Sieve of Eratosthenes, mark from `i*i`	O(n log log n)
"nth number divisible by a/b/c"	Binary search on answer + inclusion-exclusion (LCM)	O(log range)
"check if n is power of 2/4"	Bit trick `n & (n-1) == 0` (see bit_manipulation)	O(1)

Additional Reference Problems

#29 Divide Two Integers

Description: Divide two integers `dividend` and `divisor` without using multiplication, division, or mod, truncating toward zero. Clamp the result to the 32-bit signed range.
Intuition: Repeated subtraction is too slow, so subtract the largest doubling of the divisor that still fits — this is long division in binary. Work in negatives so `Integer.MIN_VALUE` cannot overflow.
Variation: Exponential search of the divisor — double `divisor` and the matching quotient bit until it would exceed the remaining dividend.

```
class Solution {
    public int divide(int dividend, int divisor) {
        if (dividend == Integer.MIN_VALUE && divisor == -1) {
            return Integer.MAX_VALUE;                 // ← VARIATION: only overflow case, clamp
        }
        boolean negative = (dividend < 0) != (divisor < 0);
        long a = Math.abs((long) dividend), b = Math.abs((long) divisor);  // long avoids overflow
        long result = 0;
        while (a >= b) {
            long temp = b, multiple = 1;
            while (a >= (temp << 1)) {                // ← VARIATION: double divisor while it fits
                temp <<= 1;
                multiple <<= 1;
            }
            a -= temp;
            result += multiple;
        }
        return negative ? (int) -result : (int) result;
    }
}
```

Time O(log n * log n) | **Space** O(1)

#59 Spiral Matrix II

Description: Given `n`, generate an `n x n` matrix filled with the numbers `1` to `n*n` in spiral order.
Intuition: Walk the four borders inward, shrinking the boundary after completing each side, filling a running counter.
Variation: Layer boundaries — maintain `top/bottom/left/right` and fill right, down, left, up in turn.

```
class Solution {
    public int[][] generateMatrix(int n) {
        int[][] result = new int[n][n];
        int top = 0, bottom = n - 1, left = 0, right = n - 1, value = 1;
        while (top <= bottom && left <= right) {
            for (int j = left; j <= right; j++) {
                result[top][j] = value++;
            }
            top++;
            for (int i = top; i <= bottom; i++) {
                result[i][right] = value++;
            }
            right--;
            for (int j = right; j >= left; j--) {
                result[bottom][j] = value++;
            }
            bottom--;
            for (int i = bottom; i >= top; i--) {
                result[i][left] = value++;
            }
            left++;
        }
        return result;
    }
}
```

Time O(n^2) | **Space** O(1) extra

#66 Plus One

Description: Given a large integer represented as an array of digits (most significant first), increment it by one and return the resulting digit array.
Intuition: Add one from the rightmost digit; a digit below 9 absorbs the carry and we're done, otherwise it becomes 0 and the carry propagates. If every digit was 9 we need one extra leading digit.
Variation: Early return — the moment a digit is incremented without rolling over, return; only an all-9 array needs a longer result.

```java
class Solution {
    public int[] plusOne(int[] digits) {
        for (int i = digits.length - 1; i >= 0; i--) {
            if (digits[i] < 9) {                 // ← VARIATION: no carry → done early
                digits[i]++;
                return digits;
            }
            digits[i] = 0;
        }
        int[] result = new int[digits.length + 1];
        result[0] = 1;                           // all nines → leading 1, rest already 0
        return result;
    }
}
```

Time O(n) | **Space** O(1) extra (O(n) only for all-nines)

#118 Pascal's Triangle

Description: Given `numRows`, generate the first `numRows` rows of Pascal's triangle.
Intuition: Each interior entry is the sum of the two entries above it; the edges are always 1.
Variation: Row-by-row build — `row[j] = previous[j-1] + previous[j]`.

```java
class Solution {
    public List<List<Integer>> generate(int numRows) {
        List<List<Integer>> result = new ArrayList<>();
        for (int i = 0; i < numRows; i++) {
            List<Integer> row = new ArrayList<>();
            for (int j = 0; j <= i; j++) {
                if (j == 0 || j == i) {
                    row.add(1);                  // edges are 1
                } else {
                    row.add(result.get(i - 1).get(j - 1) + result.get(i - 1).get(j));  // ← VARIATION: sum of two above
                }
            }
            result.add(row);
        }
        return result;
    }
}
```

Time O(numRows^2) | **Space** O(numRows^2)

#119 Pascal's Triangle II

Description: Given a row index `rowIndex`, return that row of Pascal's triangle using only O(rowIndex) extra space.
Intuition: Update a single row in place; iterating right-to-left lets each entry read its left neighbor before that neighbor is overwritten.
Variation: In-place reverse update — `row[j] += row[j-1]` scanning from the right.

```java
class Solution {
    public List<Integer> getRow(int rowIndex) {
        Integer[] row = new Integer[rowIndex + 1];
        Arrays.fill(row, 1);
        for (int i = 2; i <= rowIndex; i++) {
            for (int j = i - 1; j > 0; j--) {        // ← VARIATION: right-to-left in-place update
                row[j] += row[j - 1];
            }
        }
        return Arrays.asList(row);
    }
}
```

Time O(rowIndex^2) | **Space** O(rowIndex)

#149 Max Points on a Line

Description: Given an array of points on a plane, return the maximum number of points lying on the same straight line.
Intuition: Fix one point as the anchor; every other point defines a slope from it, and collinear points share that slope. Count the most common slope per anchor, using a reduced integer fraction as the key to avoid floating-point error.
Variation: Slope histogram per anchor — key on `(dy/g, dx/g)` reduced by gcd, with sign normalization.

```
class Solution {
    public int maxPoints(int[][] points) {
        int n = points.length;
        if (n <= 2) {
            return n;
        }
        int result = 1;
        for (int i = 0; i < n; i++) {
            Map<String, Integer> slopes = new HashMap<>();
            for (int j = 0; j < n; j++) {
                if (i == j) {
                    continue;
                }
                int dx = points[j][0] - points[i][0];
                int dy = points[j][1] - points[i][1];
                int g = gcd(dx, dy);
                dx /= g;
                dy /= g;
                if (dx < 0 || (dx == 0 && dy < 0)) {  // normalize sign for a canonical key
                    dx = -dx;
                    dy = -dy;
                }
                String key = dy + "/" + dx;           // ← VARIATION: reduced fraction as slope key
                slopes.merge(key, 1, Integer::sum);
                result = Math.max(result, slopes.get(key) + 1);
            }
        }
        return result;
    }
    private int gcd(int x, int y) {
        return y == 0 ? x : gcd(y, x % y);
    }
}
```

Time O(n^2) | **Space** O(n)

#172 Factorial Trailing Zeroes

Description: Given an integer `n`, return the number of trailing zeroes in `n!`.
Intuition: A trailing zero comes from a factor of 10 = 2*5, and factors of 2 are far more plentiful than factors of 5, so just count the factors of 5. Multiples of 25 contribute an extra 5, of 125 another, and so on.
Variation: Count factors of 5 — sum `n/5 + n/25 + n/125 + ...` via `n /= 5`.

```
class Solution {
    public int trailingZeroes(int n) {
        int result = 0;
        while (n > 0) {
            n /= 5;                  // ← VARIATION: accumulate factors of 5 at each power
            result += n;
        }
        return result;
    }
}
```

Time O(log n) | **Space** O(1)

#202 Happy Number

Description: A happy number reaches 1 by repeatedly replacing it with the sum of the squares of its digits; return whether `n` is happy.
Intuition: The sequence either hits 1 or enters a cycle, so this is cycle detection — use Floyd's slow/fast pointers over the "sum of digit squares" transformation.
Variation: Floyd cycle detection on a number transform — `slow = f(slow)`, `fast = f(f(fast))`.

```
class Solution {
    public boolean isHappy(int n) {
        int slow = n, fast = n;
        do {
            slow = squareSum(slow);              // ← VARIATION: one step
            fast = squareSum(squareSum(fast));   // two steps
        } while (slow != fast);
        return slow == 1;
    }
    private int squareSum(int n) {
        int sum = 0;
        while (n > 0) {
            int digit = n % 10;
            n /= 10;
            sum += digit * digit;
        }
        return sum;
    }
}
```

Time O(log n) per step | **Space** O(1)

#223 Rectangle Area

Description: Given the coordinates of two axis-aligned rectangles, return the total area they cover (counting the overlap once).
Intuition: Total area is the sum of both areas minus the overlap; the overlap is itself a rectangle whose width and height are the intersections of the projections onto each axis (zero if they don't intersect).
Variation: Inclusion-exclusion of areas — overlap width = `min(rights) - max(lefts)` clamped at 0.

```
class Solution {
    public int computeArea(int ax1, int ay1, int ax2, int ay2, int bx1, int by1, int bx2, int by2) {
        long areaA = (long) (ax2 - ax1) * (ay2 - ay1);
        long areaB = (long) (bx2 - bx1) * (by2 - by1);
        long overlapWidth = Math.max(0, Math.min(ax2, bx2) - Math.max(ax1, bx1));
        long overlapHeight = Math.max(0, Math.min(ay2, by2) - Math.max(ay1, by1));
        long overlap = overlapWidth * overlapHeight;          // ← VARIATION: intersection area
        return (int) (areaA + areaB - overlap);
    }
}
```

Time O(1) | **Space** O(1)

#233 Number of Digit One

Description: Given an integer `n`, count the total number of digit `1` appearing in all non-negative integers from 0 to `n`.
Intuition: Count contributions of the digit 1 at each place value separately. For a given place, the count depends on the higher digits, the current digit, and the lower digits, which split cleanly into full cycles plus a partial cycle.
Variation: Per-place digit counting — for each power-of-ten place, `count += high*place + adjust(cur, low, place)`.

```
class Solution {
    public int countDigitOne(int n) {
        long result = 0;
        for (long place = 1; place <= n; place *= 10) {       // ← VARIATION: examine each decimal place
            long high = n / (place * 10);
            long cur = (n / place) % 10;
            long low = n % place;
            if (cur == 0) {
                result += high * place;
            } else if (cur == 1) {
                result += high * place + low + 1;
            } else {
                result += (high + 1) * place;
            }
        }
        return (int) result;
    }
}
```

Time O(log n) | **Space** O(1)

#243 Shortest Word Distance

Description: Given an array of words and two distinct words, return the shortest distance (in indices) between any occurrence of the two words.
Intuition: A single pass tracking the most recent index of each target word suffices; whenever both have been seen, the gap between the two latest positions is a candidate minimum.
Variation: Two latest-index trackers — update `result` whenever both indices are valid.

```
class Solution {
    public int shortestDistance(String[] wordsDict, String word1, String word2) {
        int index1 = -1, index2 = -1, result = Integer.MAX_VALUE;
        for (int i = 0; i < wordsDict.length; i++) {
            if (wordsDict[i].equals(word1)) {
                index1 = i;
            } else if (wordsDict[i].equals(word2)) {
                index2 = i;
            }
            if (index1 >= 0 && index2 >= 0) {                 // ← VARIATION: both seen → candidate gap
                result = Math.min(result, Math.abs(index1 - index2));
            }
        }
        return result;
    }
}
```

Time O(n) | **Space** O(1)

#258 Add Digits

Description: Repeatedly add all the digits of a non-negative integer until the result has a single digit; return it (the digital root).
Intuition: The digital root has a closed form derived from numbers being congruent to their digit sum modulo 9: the answer is `0` for 0, otherwise `1 + (n−1) % 9`.
Variation: Digital root formula — no loop, just `1 + (num - 1) % 9`.

```
class Solution {
    public int addDigits(int num) {
        if (num == 0) {
            return 0;
        }
        return 1 + (num - 1) % 9;        // ← VARIATION: closed-form digital root (mod 9)
    }
}
```

Time O(1) | **Space** O(1)

#263 Ugly Number

Description: An ugly number has no prime factors other than 2, 3, or 5; return whether `n` is ugly.
Intuition: Divide out all factors of 2, 3, and 5; what remains is 1 exactly when no other prime factor exists. Non-positive numbers are never ugly.
Variation: Factor stripping — repeatedly divide by each of 2, 3, 5 and check the remainder is 1.

```
class Solution {
    public boolean isUgly(int n) {
        if (n <= 0) {
            return false;
        }
        for (int factor : new int[]{2, 3, 5}) {
            while (n % factor == 0) {        // ← VARIATION: strip out each allowed prime factor
                n /= factor;
            }
        }
        return n == 1;
    }
}
```

Time O(log n) | **Space** O(1)

#292 Nim Game

Description: You and an opponent alternate removing 1 to 3 stones; whoever takes the last stone wins. Given `n` stones and you move first, return whether you can win.
Intuition: If `n` is a multiple of 4 you always lose, because whatever you take (1-3) the opponent can complete a group of four, keeping `n` a multiple of 4 on your turn until 0.
Variation: Multiple-of-four loss — win exactly when `n % 4 != 0`.

```
class Solution {
    public boolean canWinNim(int n) {
        return (n % 4) != 0;            // ← VARIATION: lose iff n divisible by 4
    }
}
```

Time O(1) | **Space** O(1)

#319 Bulb Switcher

Description: There are `n` bulbs initially off; in round `i` you toggle every `i`-th bulb. Return how many bulbs are on after `n` rounds.
Intuition: Bulb `k` is toggled once per divisor of `k`, ending on only if it has an odd number of divisors — which happens exactly for perfect squares. The count of perfect squares up to `n` is `floor(sqrt(n))`.
Variation: Perfect-square count — answer is `(int) sqrt(n)`.

```
class Solution {
    public int bulbSwitch(int n) {
        return (int) Math.sqrt(n);       // ← VARIATION: only perfect-square positions stay on
    }
}
```

Time O(1) | **Space** O(1)

#326 Power of Three

Description: Given an integer `n`, return whether it is a power of three.
Intuition: Within the 32-bit range the largest power of three is `3^19 = 1162261467`; any power of three must divide it exactly, so a single divisibility

test works.
Variation: Max-power divisibility — `n > 0 && 1162261467 % n == 0`.

```
class Solution {
    public boolean isPowerOfThree(int n) {
        return n > 0 && 1162261467 % n == 0;   // ← VARIATION: divides the largest 32-bit power of 3
    }
}
```

Time O(1) | **Space** O(1)

#357 Count Numbers with Unique Digits

Description: Given `n`, count all numbers `x` with `0 <= x < 10^n` that have no repeated digits.
Intuition: Count by length: the first digit has 9 choices (1-9), the next 9 (any but the first), then 8, 7, ..., multiplying as digits are added. Sum these counts across lengths 1 to n, plus the single number 0.
Variation: Permutation counting per length — `9 * 9 * 8 * ...` accumulated.

```
class Solution {
    public int countNumbersWithUniqueDigits(int n) {
        if (n == 0) {
            return 1;
        }
        int result = 10;                 // lengths 0..1 (includes 0)
        int uniqueOfLength = 9;
        int availableDigits = 9;
        for (int i = 2; i <= n && availableDigits > 0; i++) {
            uniqueOfLength *= availableDigits;   // ← VARIATION: one fewer digit choice each place
            availableDigits--;
            result += uniqueOfLength;
        }
        return result;
    }
}
```

Time O(n) | **Space** O(1)

#365 Water and Jug Problem

Description: Given two jugs of capacities `x` and `y` and a target `target`, determine if you can measure exactly `target` liters using fill/empty/pour operations.
Intuition: Any amount measurable is a non-negative integer combination of `x` and `y`, which by Bezout's identity is exactly the multiples of `gcd(x, y)`. So the target must be reachable in total capacity and divisible by the gcd.
Variation: Bezout / gcd test — `target % gcd(x, y) == 0` with `target <= x + y`.

```
class Solution {
    public boolean canMeasureWater(int x, int y, int target) {
        if (target > x + y) {
            return false;
        }
        if (target == 0) {
            return true;
        }
        return target % gcd(x, y) == 0;      // ← VARIATION: reachable iff divisible by gcd (Bezout)
    }
    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```

Time O(log(min(x,y))) | **Space** O(1)

#367 Valid Perfect Square

Description: Given a positive integer `num`, return whether it is a perfect square, without using any built-in sqrt function.
Intuition: The square root lies in `[1, num]` and squaring is monotonic, so binary search the candidate root and compare its square (in `long` to avoid overflow) against `num`.
Variation: Binary search on the root — midpoint `k = i + (j−i)/2`, compare `k*k` to `num`.

```
class Solution {
    public boolean isPerfectSquare(int num) {
        long i = 1, j = num;
        while (i <= j) {
            long k = i + (j − i) / 2;        // ← VARIATION: binary search the root
            long square = k * k;             // long guards overflow
            if (square == num) {
                return true;
            } else if (square < num) {
                i = k + 1;
            } else {
                j = k − 1;
            }
        }
        return false;
    }
}
```

Time O(log n) | **Space** O(1)

#397 Integer Replacement

Description: Given a positive integer `n`, each step you may replace it with `n/2` if even, or `n+1` / `n−1` if odd; return the minimum steps to reach 1.
Intuition: Halving is always best when possible. For odd `n`, choosing `+1` vs `−1` should expose more trailing zeros to halve next; `n == 3` is the special case where `−1` is better.
Variation: Greedy on the last two bits — for odd `n != 3`, add 1 when `(n & 2) != 0` else subtract 1.

```
class Solution {
    public int integerReplacement(int n) {
        long value = n;                 // long: n+1 may overflow int at Integer.MAX_VALUE
        int result = 0;
        while (value != 1) {
            if ((value & 1) == 0) {
                value >>= 1;            // even → halve
            } else if (value == 3 || (value & 2) == 0) {
                value−−;                // ← VARIATION: clearing a single trailing 1 bit
            } else {
                value++;                // ← VARIATION: create more trailing zeros
            }
            result++;
        }
        return result;
    }
}
```

Time O(log n) | **Space** O(1)

#398 Random Pick Index

Description: Given an array `nums`, implement `pick(target)` returning a uniformly random index where `nums[index] == target`.
Intuition: Reservoir sampling lets us pick uniformly in one pass without storing all matching indices: the `k`-th matching element replaces the current pick with probability `1/k`.
Variation: Reservoir sampling of size 1 — keep the index with probability `1/count` as matches stream by.

```
class Solution {
    private int[] nums;
    private Random random = new Random();
    public Solution(int[] nums) {
        this.nums = nums;
    }
    public int pick(int target) {
        int result = -1, count = 0;
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] == target) {
                count++;
                if (random.nextInt(count) == 0) {    // ← VARIATION: reservoir, replace with prob 1/count
                    result = i;
                }
            }
        }
        return result;
    }
}
```

Time O(n) per pick | **Space** O(1)

#400 Nth Digit

Description: Given `n`, return the `n`-th digit in the infinite sequence of concatenated positive integers `123456789101112...`.
Intuition: Numbers group by digit-length: there are `9*10^(d-1)` numbers with `d` digits, contributing `d * 9 * 10^(d-1)` digits. Subtract whole groups to find the length, locate the exact number, then index into it.
Variation: Length-bucket walk — subtract `count * digitLength` until `n` falls inside the current bucket.

```
class Solution {
    public int findNthDigit(int n) {
        long digitLength = 1, count = 9, start = 1;
        while (n > digitLength * count) {            // ← VARIATION: skip whole digit-length buckets
            n -= digitLength * count;
            digitLength++;
            count *= 10;
            start *= 10;
        }
        long number = start + (n - 1) / digitLength; // the number holding the target digit
        int indexInNumber = (int) ((n - 1) % digitLength);
        return Long.toString(number).charAt(indexInNumber) - '0';
    }
}
```

Time O(log n) | **Space** O(1)

#412 Fizz Buzz

Description: Return a list of strings for `1..n` where multiples of 3 become "Fizz", multiples of 5 become "Buzz", multiples of both become "FizzBuzz", and others the number itself.
Intuition: Build each entry by concatenating "Fizz" and/or "Buzz" based on divisibility; if neither applies, use the number's string.
Variation: Concatenate divisibility tags — append "Fizz" on `%3`, "Buzz" on `%5`.

```java
class Solution {
    public List<String> fizzBuzz(int n) {
        List<String> result = new ArrayList<>();
        for (int i = 1; i <= n; i++) {
            StringBuilder builder = new StringBuilder();
            if (i % 3 == 0) {
                builder.append("Fizz");
            }
            if (i % 5 == 0) {
                builder.append("Buzz");          // ← VARIATION: tags concatenate for the both-case
            }
            if (builder.length() == 0) {
                builder.append(i);
            }
            result.add(builder.toString());
        }
        return result;
    }
}
```

Time O(n) | **Space** O(1) extra

#441 Arranging Coins

Description: You have `n` coins to form a staircase where the `k` -th row has exactly `k` coins; return the number of complete rows.
Intuition: After `k` complete rows you've used `k(k+1)/2` coins, so the answer is the largest `k` with `k(k+1)/2 <= n`. Binary search this `k` (or solve the quadratic).
Variation: Binary search on rows — compare triangular number `k(k+1)/2` against `n` using `long`.

```java
class Solution {
    public int arrangeCoins(int n) {
        long i = 1, j = n;
        while (i <= j) {
            long k = i + (j - i) / 2;            // ← VARIATION: binary search row count
            long used = k * (k + 1) / 2;         // long guards overflow
            if (used == n) {
                return (int) k;
            } else if (used < n) {
                i = k + 1;
            } else {
                j = k - 1;
            }
        }
        return (int) j;
    }
}
```

Time O(log n) | **Space** O(1)

#453 Minimum Moves to Equal Array Elements

Description: In one move you increment `n-1` elements of the array by 1; return the minimum moves to make all elements equal.
Intuition: Incrementing all but one is equivalent (for the purpose of equalizing) to decrementing a single element by 1. So the answer is the total amount each element exceeds the minimum: `sum - n * min`.
Variation: Relative-decrement reframing — total moves = `sum(nums) - n * min(nums)`.

```java
class Solution {
    public int minMoves(int[] nums) {
        int min = Integer.MAX_VALUE;
        long sum = 0;
        for (int num : nums) {
            sum += num;
            min = Math.min(min, num);
        }
        return (int) (sum - (long) nums.length * min);   // ← VARIATION: equivalent to decrementing toward min
    }
}
```

Time O(n) | **Space** O(1)

#458 Poor Pigs

Description: With `buckets` buckets one of which is poisoned, `minutesToDie` for poison to act, and `minutesToTest` total time, return the minimum pigs needed to find the poisoned bucket.
Intuition: Each pig can be tested `t = minutesToTest / minutesToDie` times, so it has `t + 1` distinguishable states (died after round 1..t, or survived). With `p` pigs we cover `(t+1)^p` buckets, so we need the smallest `p` with `(t+1)^p >= buckets`.
Variation: Base-(tests+1) counting — `p = ceil(log_{t+1}(buckets))`.

```java
class Solution {
    public int poorPigs(int buckets, int minutesToDie, int minutesToTest) {
        int statesPerPig = minutesToTest / minutesToDie + 1;   // rounds + survive state
        int result = 0;
        long reach = 1;
        while (reach < buckets) {
            reach *= statesPerPig;               // ← VARIATION: each pig multiplies reachable buckets
            result++;
        }
        return result;
    }
}
```

Time O(log buckets) | **Space** O(1)

#470 Implement Rand10() Using Rand7()

Description: Given `rand7()` uniform in [1,7], implement `rand10()` uniform in [1,10].
Intuition: Two calls form a uniform value in [1,49]; reject anything above 40 and map the remaining 40 outcomes evenly onto [1,10]. Rejection keeps the distribution exactly uniform.
Variation: Rejection sampling on a 7x7 grid — accept index `< 40`, return `index % 10 + 1`.

```java
class Solution extends SolBase {
    public int rand10() {
        while (true) {
            int row = rand7();
            int col = rand7();
            int index = (row - 1) * 7 + col;     // uniform in [1,49]
            if (index <= 40) {                   // ← VARIATION: reject > 40 to stay uniform
                return (index - 1) % 10 + 1;
            }
        }
    }
}
```

Time O(1) expected | **Space** O(1)

#492 Construct the Rectangle

Description: Given a target `area`, return the dimensions `[L, W]` with `L >= W`, `L * W == area`, and `L - W` minimized.
Intuition: The most square-like factor pair minimizes the difference, so start `W` at `floor(sqrt(area))` and walk down until it divides `area`.
Variation: Search down from sqrt — first divisor `W <= sqrt(area)` gives the closest pair.

```
class Solution {
    public int[] constructRectangle(int area) {
        int width = (int) Math.sqrt(area);
        while (area % width != 0) {              // ← VARIATION: nearest divisor at or below sqrt
            width--;
        }
        return new int[]{area / width, width};
    }
}
```

Time O(sqrt(area)) | **Space** O(1)

#495 Teemo Attacking

Description: Given sorted attack `timeSeries` and a poison `duration`, return the total time the target is poisoned (overlaps counted once).
Intuition: Each attack would add `duration`, but if the next attack comes before the current poison ends, only the gap between attacks counts. Sum the capped gaps and add a full duration for the last attack.
Variation: Capped gaps — add `min(gap, duration)` between consecutive attacks.

```
class Solution {
    public int findPoisonedDuration(int[] timeSeries, int duration) {
        if (timeSeries.length == 0) {
            return 0;
        }
        int result = 0;
        for (int i = 1; i < timeSeries.length; i++) {
            result += Math.min(timeSeries[i] - timeSeries[i - 1], duration);  // ← VARIATION: overlap-capped gap
        }
        return result + duration;            // last attack always contributes full duration
    }
}
```

Time O(n) | **Space** O(1)

#528 Random Pick with Weight

Description: Given an array `w` of weights, implement `pickIndex()` returning index `i` with probability proportional to `w[i]`.
Intuition: Build prefix sums so the cumulative weights partition `[1, total]` into intervals; draw a random target in that range and binary-search the first prefix sum reaching it.
Variation: Prefix-sum + binary search — find leftmost prefix `>= target`.

```java
class Solution {
    private int[] prefix;
    private int total;
    private Random random = new Random();
    public Solution(int[] w) {
        prefix = new int[w.length];
        int sum = 0;
        for (int i = 0; i < w.length; i++) {
            sum += w[i];
            prefix[i] = sum;
        }
        total = sum;
    }
    public int pickIndex() {
        int target = random.nextInt(total) + 1;   // in [1, total]
        int i = 0, j = prefix.length - 1;
        while (i < j) {
            int k = i + (j - i) / 2;               // ← VARIATION: leftmost prefix >= target
            if (prefix[k] < target) {
                i = k + 1;
            } else {
                j = k;
            }
        }
        return i;
    }
}
```

Time O(log n) per pick, O(n) build | **Space** O(n)

#593 Valid Square

Description: Given four points, return whether they form a valid square (positive area).
Intuition: Among the six pairwise squared distances of a square there are exactly two distinct values: four equal sides and two equal (larger) diagonals, with the diagonal twice the side. Use squared distances to stay in integers.
Variation: Two-distinct-distance check — collect six squared distances; require exactly two values, the smaller nonzero, the larger double it.

```java
class Solution {
    public boolean validSquare(int[] p1, int[] p2, int[] p3, int[] p4) {
        long[] distances = {
            dist(p1, p2), dist(p1, p3), dist(p1, p4),
            dist(p2, p3), dist(p2, p4), dist(p3, p4)
        };
        Arrays.sort(distances);
        if (distances[0] == 0) {                 // degenerate (coincident points)
            return false;
        }
        return distances[0] == distances[3]      // ← VARIATION: four equal sides
            && distances[4] == distances[5]      // two equal diagonals
            && distances[4] == 2 * distances[0]; // diagonal^2 = 2 * side^2
    }
    private long dist(int[] a, int[] b) {
        long dx = a[0] - b[0], dy = a[1] - b[1];
        return dx * dx + dy * dy;
    }
}
```

Time O(1) | **Space** O(1)

#598 Range Addition II

Description: Given an `m x n` matrix of zeros and operations each incrementing the top-left `a x b` submatrix, return how many cells hold the maximum value.

Intuition: Every operation always covers cell (0,0), so the maximum value sits in the intersection of all operation rectangles — its area is the product of the smallest row bound and smallest column bound.
Variation: Intersection area of all ops — `min(all a) * min(all b)`.

```
class Solution {
    public int maxCount(int m, int n, int[][] ops) {
        int minRow = m, minCol = n;
        for (int[] op : ops) {
            minRow = Math.min(minRow, op[0]);    // ← VARIATION: shrink to common intersection
            minCol = Math.min(minCol, op[1]);
        }
        return minRow * minCol;
    }
}
```

Time O(ops) | **Space** O(1)

#633 Sum of Square Numbers

Description: Given a non-negative integer `c`, decide whether there exist non-negative integers `a, b` with `a*a + b*b == c`.
Intuition: Squeeze two pointers from `0` and `floor(sqrt(c))`: if the squared sum is too small move the low pointer up, too large move the high pointer down.
Variation: Two pointers on squares — `a` from 0 up, `b` from sqrt(c) down.

```
class Solution {
    public boolean judgeSquareSum(int c) {
        long a = 0, b = (long) Math.sqrt(c);
        while (a <= b) {
            long sum = a * a + b * b;            // ← VARIATION: converging squared sum
            if (sum == c) {
                return true;
            } else if (sum < c) {
                a++;
            } else {
                b--;
            }
        }
        return false;
    }
}
```

Time O(sqrt(c)) | **Space** O(1)

#656 Coin Path

Description: Given `coins` (cost per index, `-1` blocked) and a max jump `maxJump`, find the lexicographically smallest cheapest path of indices from index 0 to the last.
Intuition: Work backwards with DP: the best cost from index `i` is its cost plus the cheapest reachable next index within `maxJump`. Filling from the right and preferring the smaller next index yields the lexicographically smallest path on ties.
Variation: Backward DP with path reconstruction — `cost[i] = coins[i] + min over j in (i, i+maxJump]`, tie-break to smaller `j`.

```
class Solution {
    public List<Integer> cheapestJump(int[] coins, int maxJump) {
        int n = coins.length;
        long[] cost = new long[n];
        int[] next = new int[n];
        Arrays.fill(cost, Long.MAX_VALUE);
        Arrays.fill(next, -1);
        if (coins[n - 1] != -1) {
            cost[n - 1] = coins[n - 1];
        }
        for (int i = n - 2; i >= 0; i--) {
            if (coins[i] == -1) {
                continue;
            }
            for (int j = i + 1; j <= Math.min(n - 1, i + maxJump); j++) {
                if (cost[j] == Long.MAX_VALUE) {
                    continue;
                }
                long candidate = cost[j] + coins[i];
                if (candidate < cost[i]) {           // ← VARIATION: cheaper path; smaller j seen first ties lexicographically
                    cost[i] = candidate;
                    next[i] = j;
                }
            }
        }
        List<Integer> result = new ArrayList<>();
        if (cost[0] == Long.MAX_VALUE) {
            return result;
        }
        for (int i = 0; i != -1; i = next[i]) {
            result.add(i + 1);                       // 1-indexed path
        }
        return result;
    }
}
```

Time O(n * maxJump) | **Space** O(n)

#781 Rabbits in Forest

Description: Each surveyed rabbit reports how many other rabbits share its color; given the `answers`, return the minimum number of rabbits in the forest.
Intuition: Rabbits answering `k` form groups of size `k+1`. For each answer value, every full group of `k+1` such replies fills one color group; partial groups still require a whole `k+1` block.
Variation: Group-rounding by answer — for count `c` of answer `k`, add `ceil(c/(k+1)) * (k+1)`.

```
class Solution {
    public int numRabbits(int[] answers) {
        Map<Integer, Integer> counts = new HashMap<>();
        for (int answer : answers) {
            counts.merge(answer, 1, Integer::sum);
        }
        int result = 0;
        for (Map.Entry<Integer, Integer> entry : counts.entrySet()) {
            int groupSize = entry.getKey() + 1;
            int count = entry.getValue();
            int groups = (count + groupSize - 1) / groupSize;   // ← VARIATION: round up to whole color groups
            result += groups * groupSize;
        }
        return result;
    }
}
```

Time O(n) | **Space** O(n)

#789 Escape The Ghosts

Description: Starting at the origin with a `target`, and given `ghosts` positions, you all move simultaneously in Manhattan steps; return whether you can reach the target before any ghost catches you.
Intuition: You win iff you reach the target strictly before every ghost. Since all move optimally with Manhattan distance, compare your distance from origin to each ghost's distance to the target; if no ghost is at least as close, you escape.
Variation: Manhattan-distance race — you win when your distance to target < every ghost's distance to target.

```
class Solution {
    public boolean escapeGhosts(int[][] ghosts, int[] target) {
        int myDist = Math.abs(target[0]) + Math.abs(target[1]);
        for (int[] ghost : ghosts) {
            int ghostDist = Math.abs(ghost[0] - target[0]) + Math.abs(ghost[1] - target[1]);
            if (ghostDist <= myDist) {           // ← VARIATION: a ghost reaches target no later → caught
                return false;
            }
        }
        return true;
    }
}
```

Time O(g) | **Space** O(1)

#792 Number of Matching Subsequences

Description: Given a string `s` and an array of `words`, return how many words are subsequences of `s`.
Intuition: Rather than scan `s` per word, bucket words by their next-needed character. Sweep `s` once; each character releases its waiting words, advancing them to their next character bucket, and any word that runs out of characters is a match.
Variation: Waiting lists per next-char — advance words bucketed by the character they currently need as `s` streams.

```
class Solution {
    public int numMatchingSubseq(String s, String[] words) {
        List<int[]>[] waiting = new List[26];    // [wordIndex, positionInWord]
        for (int i = 0; i < 26; i++) {
            waiting[i] = new ArrayList<>();
        }
        for (int i = 0; i < words.length; i++) {
            waiting[words[i].charAt(0) - 'a'].add(new int[]{i, 0});
        }
        int result = 0;
        for (char c : s.toCharArray()) {
            List<int[]> current = waiting[c - 'a'];
            waiting[c - 'a'] = new ArrayList<>();
            for (int[] entry : current) {        // ← VARIATION: advance each waiting word one character
                int wordIndex = entry[0], pos = entry[1] + 1;
                if (pos == words[wordIndex].length()) {
                    result++;
                } else {
                    waiting[words[wordIndex].charAt(pos) - 'a'].add(new int[]{wordIndex, pos});
                }
            }
        }
        return result;
    }
}
```

Time O(|s| + total word length) | **Space** O(total word length)

#829 Consecutive Numbers Sum

Description: Given `n`, return the number of ways to write it as a sum of consecutive positive integers.
Intuition: A run of `k` consecutive integers starting at `a` sums to `k*a + k(k−1)/2`, so `n − k(k−1)/2` must be positive and divisible by `k`. Try

each `k` while the triangular offset stays below `n`.
Variation: Count valid run-lengths — for each `k`, valid iff `(n − k(k−1)/2) % k == 0` and positive.

```
class Solution {
    public int consecutiveNumbersSum(int n) {
        int result = 0;
        for (long k = 1; k * (k - 1) / 2 < n; k++) {     // ← VARIATION: try each run length k
            long remainder = n - k * (k - 1) / 2;
            if (remainder % k == 0) {
                result++;
            }
        }
        return result;
    }
}
```

Time O(sqrt(n)) | **Space** O(1)

#836 Rectangle Overlap

Description: Given two axis-aligned rectangles `rec1` and `rec2` as `[x1,y1,x2,y2]`, return whether they overlap in positive area.
Intuition: Two rectangles overlap iff their projections on both axes overlap; equivalently, neither is entirely to one side of the other on either axis.
Variation: Separating-axis on both projections — overlap iff `x` ranges and `y` ranges each intersect.

```
class Solution {
    public boolean isRectangleOverlap(int[] rec1, int[] rec2) {
        return rec1[0] < rec2[2] && rec2[0] < rec1[2]   // ← VARIATION: x-projections overlap
            && rec1[1] < rec2[3] && rec2[1] < rec1[3];  // y-projections overlap
    }
}
```

Time O(1) | **Space** O(1)

#858 Mirror Reflection

Description: A square room with mirrored walls has receptors at corners 0,1,2; a laser leaves the southwest corner and first travels to the east wall at height `q` (room side `p`). Return which receptor it first meets.
Intuition: Unfold the reflections so the beam travels straight; it hits a corner after going up a multiple of `p` that is also a multiple of `q`, i.e. `lcm(p,q)`. The parities of how many room-widths and room-heights that represents determine the receptor.
Variation: Parity of `lcm/p` and `lcm/q` — reduce `p,q` by gcd, then the receptor follows from which is odd/even.

```
class Solution {
    public int mirrorReflection(int p, int q) {
        int g = gcd(p, q);
        int rows = (p / g) % 2;          // ← VARIATION: parity after dividing out gcd
        int cols = (q / g) % 2;
        if (rows == 1 && cols == 1) {
            return 1;
        } else if (rows == 1 && cols == 0) {
            return 0;
        } else {
            return 2;
        }
    }
    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```

Time O(log(min(p,q))) | **Space** O(1)

#869 Reordered Power of 2

Description: Given an integer `n`, return whether some permutation of its digits (no leading zero) equals a power of 2.
Intuition: Two numbers are digit-permutations iff they have the same multiset of digits. So compute a digit-count signature of `n` and compare it against the signatures of all powers of 2 within the 32-bit range.
Variation: Digit-count signature match — compare sorted-digit fingerprint against each power of 2.

```
class Solution {
    public boolean reorderedPowerOf2(int n) {
        long target = signature(n);
        for (int power = 1; power > 0; power <<= 1) {    // ← VARIATION: compare against each power of 2
            if (signature(power) == target) {
                return true;
            }
        }
        return false;
    }
    private long signature(int n) {
        long sig = 0;
        while (n > 0) {
            sig += (long) Math.pow(10, n % 10);   // count of each digit, encoded by place
            n /= 10;
        }
        return sig;
    }
}
```

Time O(log^2 n) | **Space** O(1)

#939 Minimum Area Rectangle

Description: Given points in the plane, find the minimum area of an axis-aligned rectangle formed by four of them, or 0 if none exists.
Intuition: A rectangle is fixed by its diagonal corners; for each pair of points with different x and y, the other two corners are determined, so check whether both exist in a point set.
Variation: Diagonal-pair lookup — for each pair, test if the complementary corners are present in a hash set.

```
class Solution {
    public int minAreaRect(int[][] points) {
        Set<Integer> seen = new HashSet<>();
        for (int[] point : points) {
            seen.add(point[0] * 40001 + point[1]);   // encode coordinate as single key
        }
        int result = Integer.MAX_VALUE;
        for (int i = 0; i < points.length; i++) {
            for (int j = i + 1; j < points.length; j++) {
                if (points[i][0] != points[j][0] && points[i][1] != points[j][1]) {   // diagonal corners
                    if (seen.contains(points[i][0] * 40001 + points[j][1])
                        && seen.contains(points[j][0] * 40001 + points[i][1])) {      // ← VARIATION: other two corners present
                        int area = Math.abs(points[i][0] - points[j][0]) * Math.abs(points[i][1] - points[j][1]);
                        result = Math.min(result, area);
                    }
                }
            }
        }
        return result == Integer.MAX_VALUE ? 0 : result;
    }
}
```

Time O(n^2) | **Space** O(n)

#963 Minimum Area Rectangle II

Description: Given points in the plane, find the minimum area of any rectangle (any orientation) formed by four of them, or 0.
Intuition: A rectangle's diagonals share the same midpoint and the same length. Group point pairs by `(midpoint, diagonal length)`; any two pairs in a group are the two diagonals of a rectangle, whose sides come from the corner vectors.
Variation: Group diagonals by midpoint+length — within a group, combine pairs and compute area via vectors.

```
class Solution {
    public double minAreaFreeRect(int[][] points) {
        Map<String, List<int[]>> groups = new HashMap<>();
        int n = points.length;
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                int cx = points[i][0] + points[j][0];      // 2*midpoint (kept as int)
                int cy = points[i][1] + points[j][1];
                long len = (long) (points[i][0] - points[j][0]) * (points[i][0] - points[j][0])
                         + (long) (points[i][1] - points[j][1]) * (points[i][1] - points[j][1]);
                String key = cx + "," + cy + "," + len;    // ← VARIATION: same midpoint + diagonal length
                groups.computeIfAbsent(key, x -> new ArrayList<>()).add(new int[]{i, j});
            }
        }
        double result = Double.MAX_VALUE;
        for (List<int[]> group : groups.values()) {
            for (int a = 0; a < group.size(); a++) {
                for (int b = a + 1; b < group.size(); b++) {
                    int p1 = group.get(a)[0], p2 = group.get(b)[0], p3 = group.get(b)[1];
                    double side1 = Math.hypot(points[p1][0] - points[p2][0], points[p1][1] - points[p2][1]);
                    double side2 = Math.hypot(points[p1][0] - points[p3][0], points[p1][1] - points[p3][1]);
                    result = Math.min(result, side1 * side2);
                }
            }
        }
        return result == Double.MAX_VALUE ? 0 : result;
    }
}
```

Time O(n^2) groups, worst O(n^2) per group | **Space** O(n^2)

#1041 Robot Bounded In Circle

Description: A robot starts at the origin facing north and repeats `instructions` ('G' forward, 'L'/'R' turn) forever; return whether it stays within some bounded circle.
Intuition: After one pass, the robot is bounded iff it returns to the origin, or it no longer faces north — a non-north heading guarantees the net displacement rotates and cancels over at most four cycles.
Variation: One-cycle check — bounded iff back at origin OR final direction != initial north.

```
class Solution {
    public boolean isRobotBounded(String instructions) {
        int x = 0, y = 0, dir = 0;               // dir 0=N,1=E,2=S,3=W
        int[][] moves = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
        for (char c : instructions.toCharArray()) {
            if (c == 'L') {
                dir = (dir + 3) % 4;
            } else if (c == 'R') {
                dir = (dir + 1) % 4;
            } else {
                x += moves[dir][0];
                y += moves[dir][1];
            }
        }
        return (x == 0 && y == 0) || dir != 0;   // ← VARIATION: origin return or non-north heading
    }
}
```

Time O(n) | **Space** O(1)

#1160 Find Words That Can Be Formed by Characters

Description: Given `words` and a string `chars`, return the total length of words that can be formed using letters of `chars` (each letter used at most as often as it appears).
Intuition: Count available letters once; a word is formable iff its own letter counts never exceed the available counts. Sum lengths of formable words.
Variation: Frequency containment — compare per-word letter counts against the `chars` budget.

```
class Solution {
    public int countCharacters(String[] words, String chars) {
        int[] budget = new int[26];
        for (char c : chars.toCharArray()) {
            budget[c - 'a']++;
        }
        int result = 0;
        for (String word : words) {
            int[] need = new int[26];
            boolean formable = true;
            for (char c : word.toCharArray()) {
                need[c - 'a']++;
                if (need[c - 'a'] > budget[c - 'a']) {   // ← VARIATION: exceeds available budget
                    formable = false;
                    break;
                }
            }
            if (formable) {
                result += word.length();
            }
        }
        return result;
    }
}
```

Time O(total characters) | **Space** O(1)

#1304 Find N Unique Integers Sum up to Zero

Description: Given `n`, return any array of `n` unique integers that sum to zero.
Intuition: Pair each positive `i` with its negation `-i`; that cancels to zero, and add a lone 0 if `n` is odd.
Variation: Symmetric pairs — emit `i` and `-i`, plus a central 0 for odd `n`.

```
class Solution {
    public int[] sumZero(int n) {
        int[] result = new int[n];
        int index = 0;
        for (int i = 1; i <= n / 2; i++) {
            result[index++] = i;                 // ← VARIATION: symmetric +i / -i pair
            result[index++] = -i;
        }
        if ((n & 1) == 1) {
            result[index] = 0;                   // odd n → central zero
        }
        return result;
    }
}
```

Time O(n) | **Space** O(1) extra

#1344 Angle Between Hands of a Clock

Description: Given an `hour` and `minutes`, return the smaller angle (in degrees) between the hour and minute hands.
Intuition: The minute hand moves 6 degrees per minute; the hour hand moves 30 degrees per hour plus 0.5 degree per minute. The answer is the absolute difference, folded to at most 180.
Variation: Per-hand angular position — minute = `6*m`, hour = `30*(h%12) + 0.5*m`, take `min(diff, 360-diff)`.

```
class Solution {
    public double angleClock(int hour, int minutes) {
        double minuteAngle = 6.0 * minutes;
        double hourAngle = 30.0 * (hour % 12) + 0.5 * minutes;   // ← VARIATION: hour hand drifts with minutes
        double diff = Math.abs(hourAngle - minuteAngle);
        return Math.min(diff, 360 - diff);
    }
}
```

Time O(1) | **Space** O(1)

#1583 Count Unhappy Friends

Description: Given `n` friends, their `preferences`, and a `pairs` assignment, count friends who are unhappy — paired with someone they prefer less than another friend who also prefers them over their own partner.
Intuition: Precompute each friend's preference rank for every other friend. A friend `x` (paired with `y`) is unhappy if some `u` ranks higher than `y` for `x`, and `u` ranks `x` higher than `u`'s own partner. Compare ranks pairwise.
Variation: Rank-matrix mutual-preference scan — `x` unhappy if exists `u` with `rank[x][u] < rank[x][y]` and `rank[u][x] < rank[u][partner[u]]`.

```
class Solution {
    public int unhappyFriends(int n, int[][] preferences, int[][] pairs) {
        int[][] rank = new int[n][n];
        for (int i = 0; i < n; i++) {
            for (int r = 0; r < preferences[i].length; r++) {
                rank[i][preferences[i][r]] = r;      // lower rank = more preferred
            }
        }
        int[] partner = new int[n];
        for (int[] pair : pairs) {
            partner[pair[0]] = pair[1];
            partner[pair[1]] = pair[0];
        }
        int result = 0;
        for (int x = 0; x < n; x++) {
            int y = partner[x];
            boolean unhappy = false;
            for (int u = 0; u < n && !unhappy; u++) {
                if (rank[x][u] < rank[x][y]) {       // ← VARIATION: x prefers u over its partner
                    if (rank[u][x] < rank[u][partner[u]]) {   // and u prefers x over its partner
                        unhappy = true;
                    }
                }
            }
            if (unhappy) {
                result++;
            }
        }
        return result;
    }
}
```

Time O(n^2) | **Space** O(n^2)

#1588 Sum of All Odd Length Subarrays

Description: Given an array `arr`, return the sum of all elements over all odd-length contiguous subarrays.
Intuition: Instead of enumerating subarrays, count how many odd-length subarrays include each index. With `i+1` choices for the left boundary and `n-i` for the right, the number of odd-length ones is computed in closed form, weighting each element.
Variation: Per-element contribution — element `i` appears in `ceil(((i+1)*(n-i))/2)` odd-length subarrays.

```
class Solution {
    public int sumOddLengthSubarrays(int[] arr) {
        int n = arr.length, result = 0;
        for (int i = 0; i < n; i++) {
            int totalSubarrays = (i + 1) * (n - i);
            int oddCount = (totalSubarrays + 1) / 2;   // ← VARIATION: odd-length share of subarrays through i
            result += oddCount * arr[i];
        }
        return result;
    }
}
```

Time O(n) | **Space** O(1)

#1646 Get Maximum in Generated Array

Description: Build array `nums` where `nums[0]=0`, `nums[1]=1`, `nums[2i]=nums[i]`, `nums[2i+1]=nums[i]+nums[i+1]`; return its maximum for size `n`.
Intuition: Generate the array directly from the recurrence, tracking the running maximum. Handle the tiny base cases for `n < 2`.
Variation: Direct recurrence fill — even index copies, odd index sums the two halves.

```
class Solution {
    public int getMaximumGenerated(int n) {
        if (n == 0) {
            return 0;
        }
        int[] nums = new int[n + 1];
        nums[1] = 1;
        int result = 1;
        for (int i = 2; i <= n; i++) {
            if ((i & 1) == 0) {
                nums[i] = nums[i / 2];                       // ← VARIATION: even index copies
            } else {
                nums[i] = nums[i / 2] + nums[i / 2 + 1];     // odd index sums neighbors
            }
            result = Math.max(result, nums[i]);
        }
        return result;
    }
}
```

Time O(n) | **Space** O(n)

#1823 Find the Winner of the Circular Game

Description: `n` friends in a circle eliminate every `k`-th friend repeatedly; return the 1-indexed winner (Josephus problem).
Intuition: The Josephus recurrence builds the survivor's position from a circle of size 1 upward: the winner in a circle of `i` is `(winner_{i-1} + k) % i`. Convert the final 0-indexed answer to 1-indexed.
Variation: Josephus recurrence — `winner = (winner + k) % i` for `i = 2..n`.

```java
class Solution {
    public int findTheWinner(int n, int k) {
        int winner = 0;                          // 0-indexed survivor for circle of size 1
        for (int i = 2; i <= n; i++) {
            winner = (winner + k) % i;           // ← VARIATION: Josephus recurrence
        }
        return winner + 1;                       // back to 1-indexed
    }
}
```

Time O(n) | **Space** O(1)

#1969 Minimum Non-Zero Product of the Array Elements

Description: For the array of all integers `1..2^p - 1`, you may swap bits between elements any number of times; return the minimum non-zero product modulo `1e9+7`.
Intuition: Pair the largest value `2^p - 1` (kept whole) with the rest: each other pair can be made `(2^p - 2)` and `1`, so the product is `(2^p - 1) * (2^p - 2)^(2^(p-1) - 1)` — compute with modular fast power, but the base of the exponent must use the true (non-reduced) count.
Variation: Pairing + modular fast power — `(max) * pow(max-1, half-1) mod M`.

```java
class Solution {
    private static final int MOD = 1_000_000_007;
    public int minNonZeroProduct(int p) {
        long max = (1L << p) - 1;                // 2^p - 1
        long exponent = (1L << (p - 1)) - 1;     // number of (max-1) factors
        long result = max % MOD * powmod((max - 1) % MOD, exponent) % MOD;  // ← VARIATION: pair-min times fast power
        return (int) result;
    }
    private long powmod(long base, long exp) {
        long result = 1;
        base %= MOD;
        while (exp > 0) {
            if ((exp & 1) == 1) {
                result = result * base % MOD;
            }
            base = base * base % MOD;
            exp >>= 1;
        }
        return result;
    }
}
```

Time O(p) | **Space** O(1)